

Solo E-Commerce Platform — Architecture Document

Document Version: 1.0
Date: 17 March 2026
Author: Solo Engineering Team
Status: Final

1. Architecture Overview

Solo E-Commerce follows a **three-tier client-server architecture** with clear separation between the presentation layer (Flutter Web), the business logic layer (NestJS API), and the data persistence layer (PostgreSQL). Communication between tiers is strictly through RESTful JSON APIs over HTTPS.

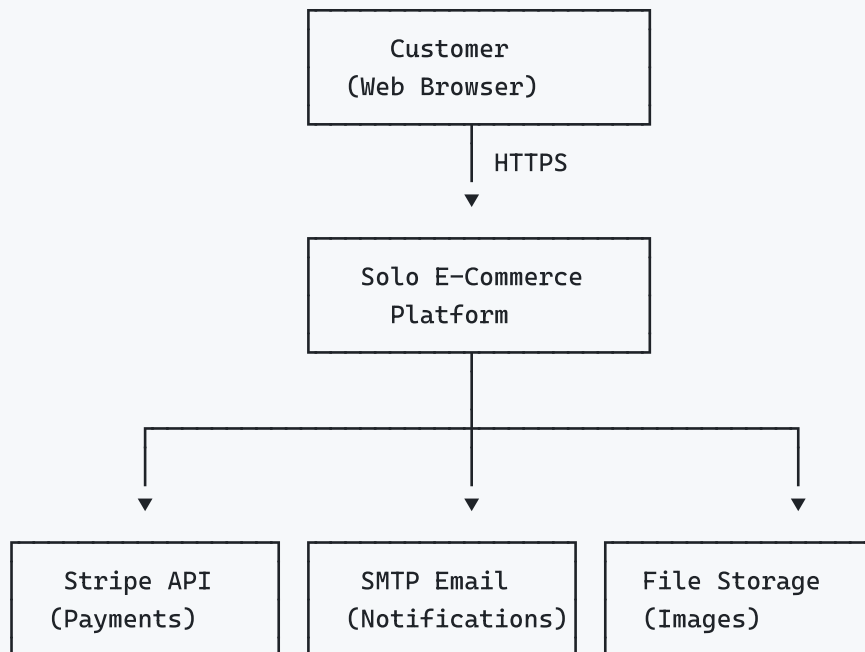
1.1 Architecture Style

Characteristic	Implementation
Pattern	Monolithic modular (backend), SPA (frontend)
API Style	RESTful with JSON payloads
State Management	Stateless backend (JWT), stateful frontend (Provider)
Data Access	ORM (Prisma) + Raw SQL for inventory
Authentication	Token-based (JWT access + refresh)
Deployment	Static frontend + API server + managed database

1.2 Design Principles

- Separation of Concerns** — Each NestJS module owns one business domain
 - Single Responsibility** — Controllers handle HTTP, Services handle logic, Prisma handles data
 - Dependency Injection** — NestJS IoC container manages all service lifecycles
 - Fail-Fast Validation** — DTO validation at controller boundary before business logic
 - Defence in Depth** — Multiple security layers (Helmet, CORS, Throttle, JWT, RBAC)
-

2. System Context Diagram



3. Component Architecture

3.1 Frontend Component Map

FLUTTER WEB APPLICATION

ENTRY

main.dart → App (MaterialApp + MultiProvider + Router)

SCREENS (Views)

Storefront

HomeScreen
HomeScreenCms
CategoryScreen
CategoryLandingScreen
ProductDetailScreen
SearchScreen
FavoritesScreen
CartScreen
CheckoutScreen
AboutUsScreen
BulkOrderScreen

Account

LoginScreen
SignupScreen
ForgotPasswordScreen
VerifyEmailScreen
MyAccountScreen
MyAddressesScreen
LoyaltyProgramScreen

Admin

AdminDashboardScreen
AdminProductsScreen
AdminCategoriesScreen
AdminBrandsScreen
AdminOrdersScreen
AdminCustomersScreen
AdminBannersScreen
AdminPromoCodesScreen
AdminLandingPagesScreen
AdminReportsScreen
AdminStripeConfigScreen
AdminVatConfigScreen

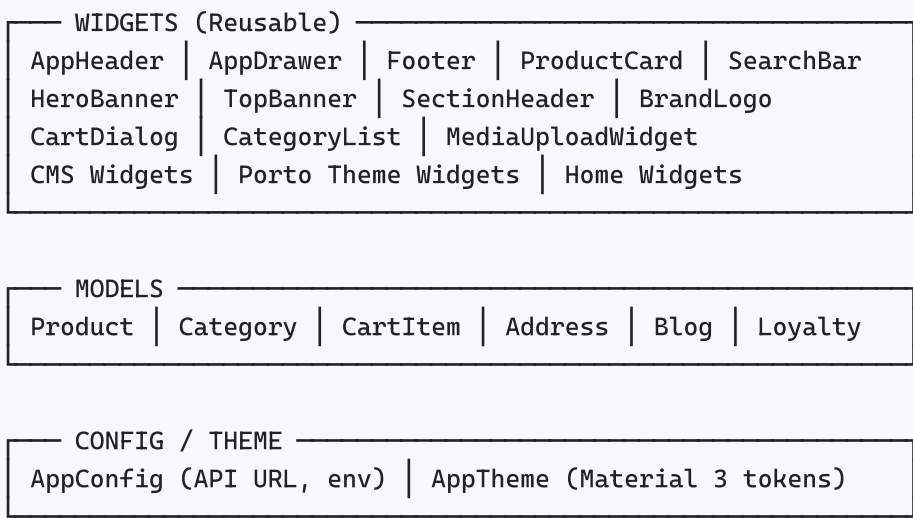
STATE (Providers)

AuthProvider | CartProvider | ProductListProvider
ProductDetailsProvider | CategoryProvider | SearchProvider
FavoritesProvider | HomeCmsProvider | HomeProvider
AccountProvider | ContentProvider

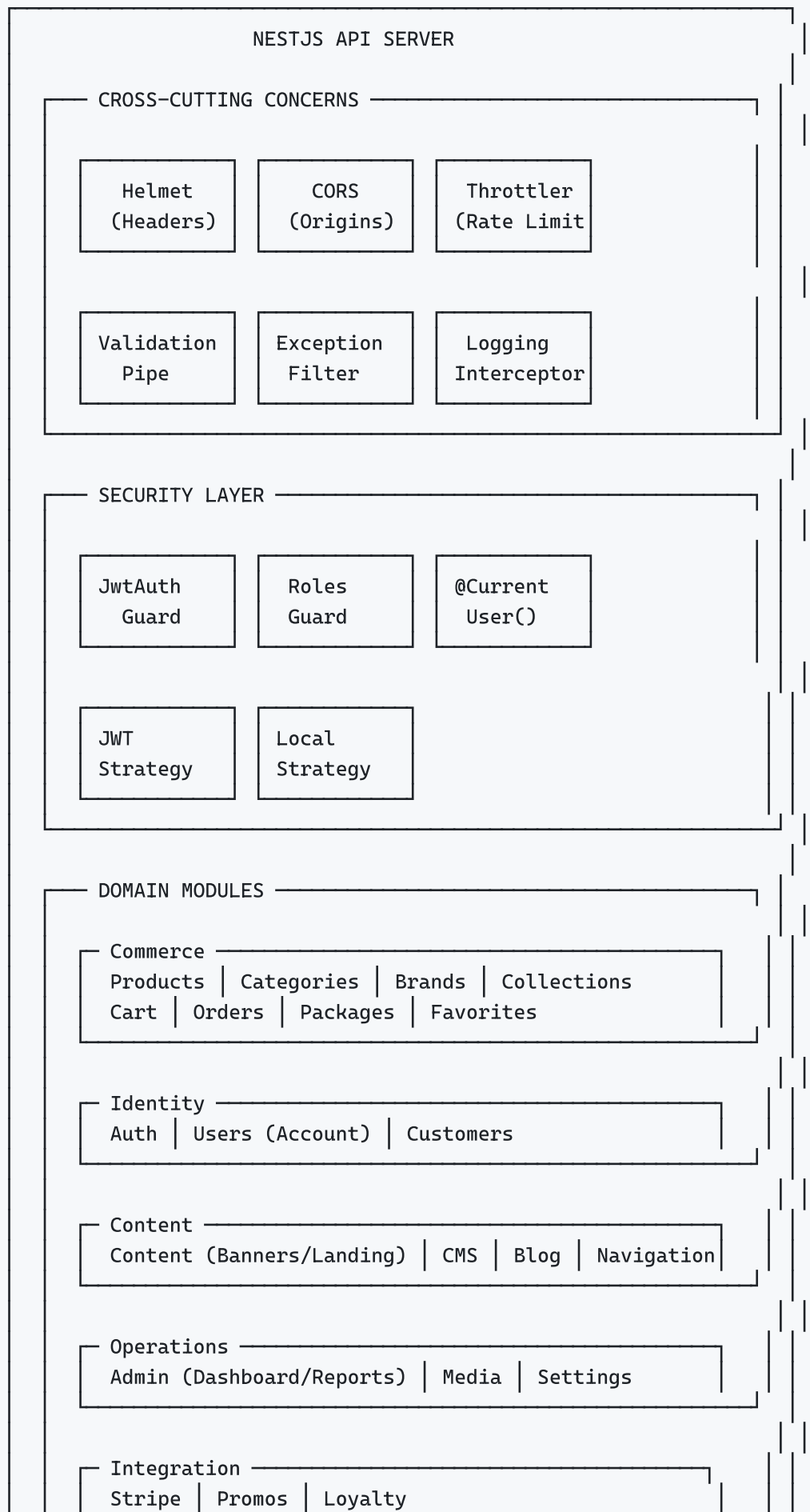
API LAYER

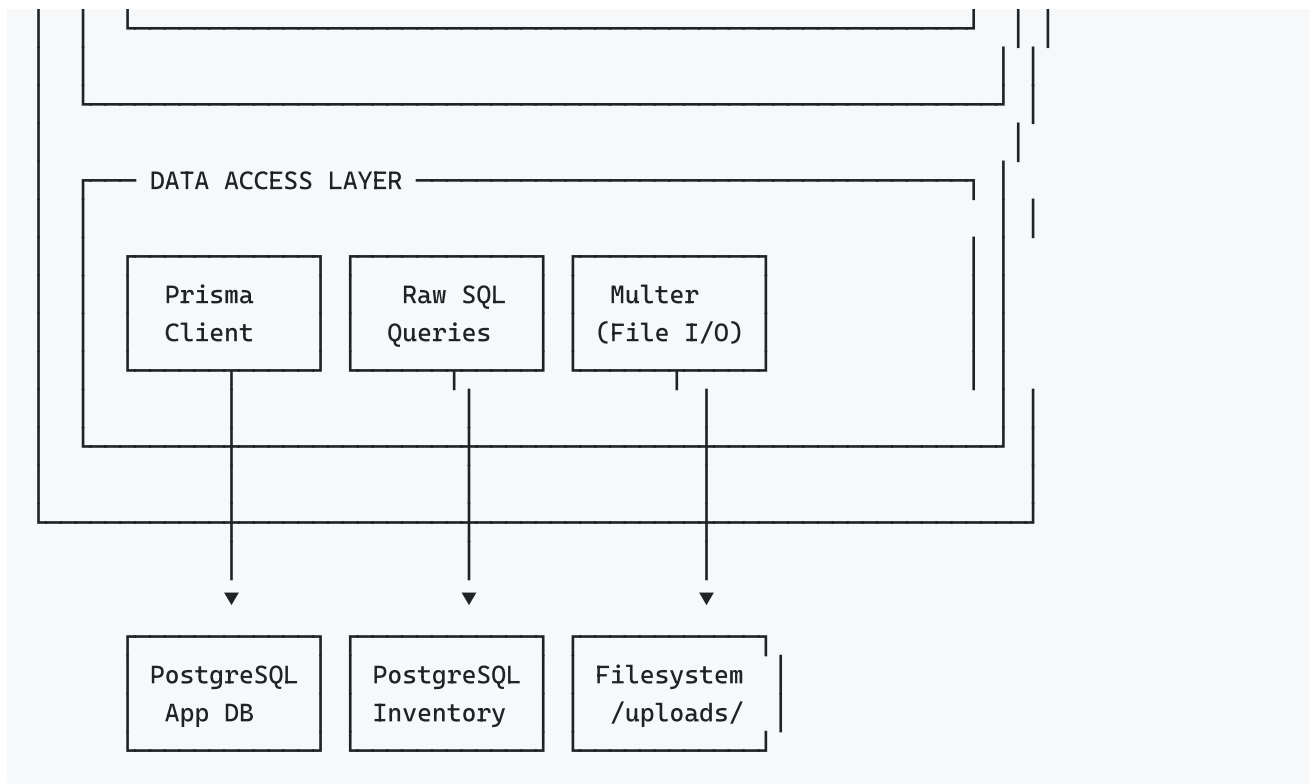
ApiClient (HTTP + Auth Interceptor + Error Handling)

AuthApi	ProductsApi	CartApi
CategoriesApi	BrandsApi	OrdersApi
FavoritesApi	ContentApi	MediaApi
AccountApi	AdminApi	CustomersApi
StripeApi	PromoCodesApi	LoyaltyApi



3.2 Backend Component Map



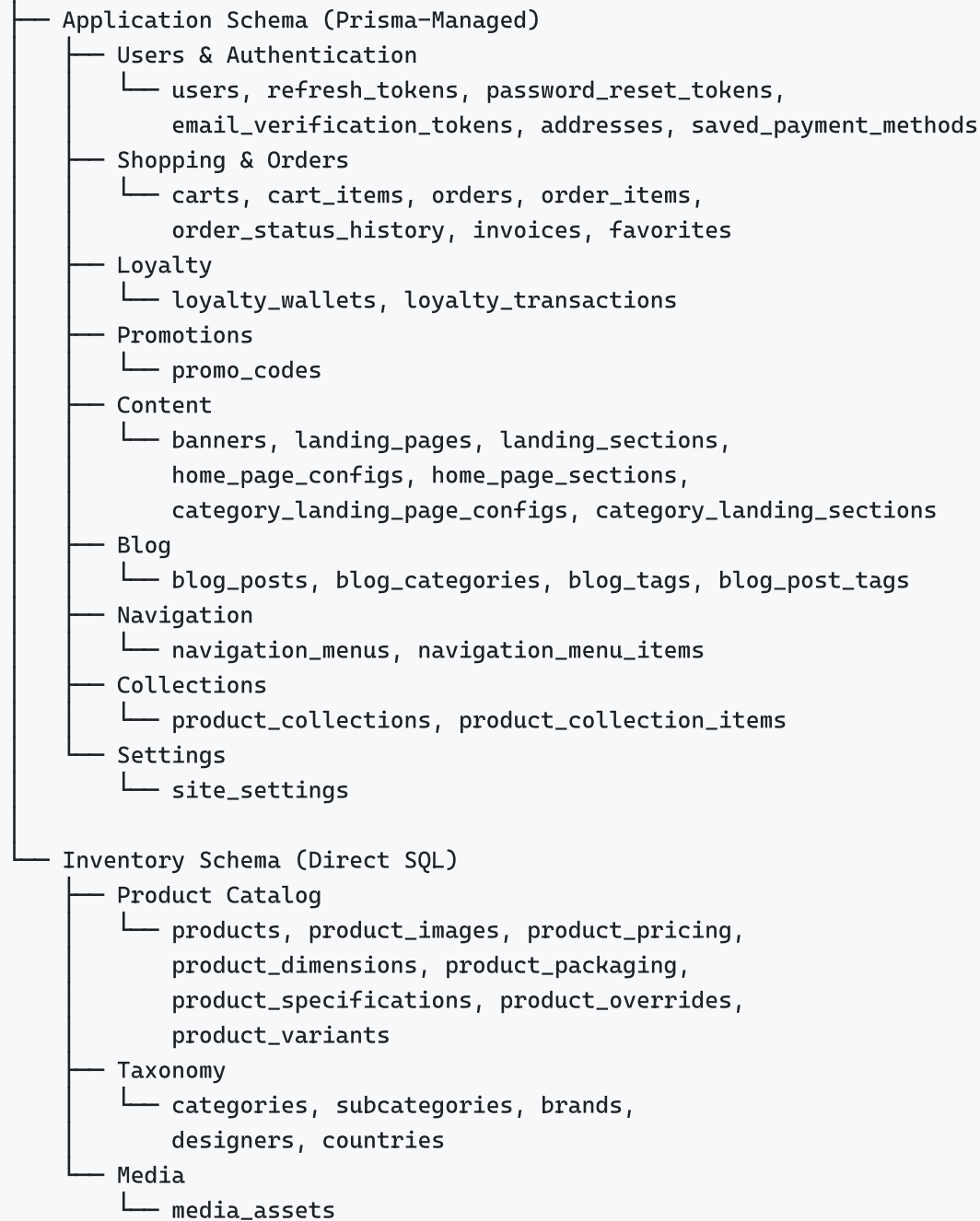


4. Data Architecture

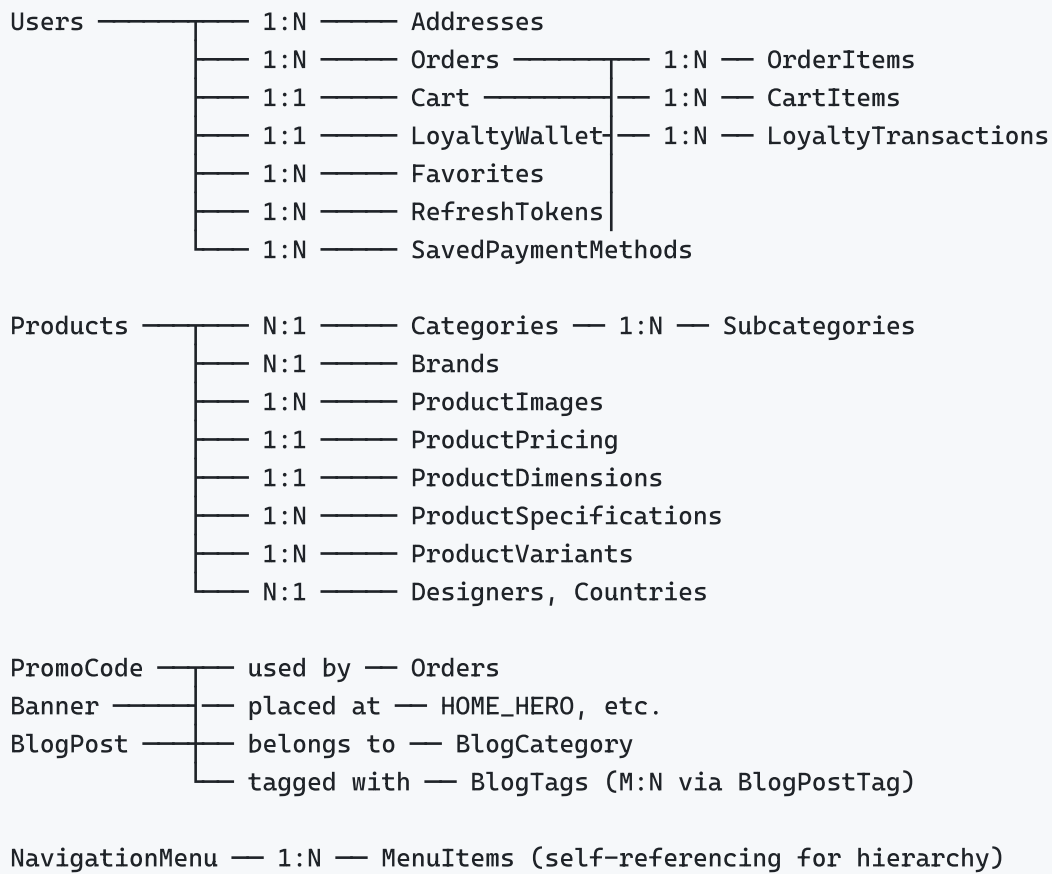
4.1 Dual-Database Strategy

The platform uses a **dual-database architecture** within a single PostgreSQL instance:

solo_ecommerce (PostgreSQL Instance)



4.2 Entity Relationship Overview



5. Security Architecture

5.1 Defence in Depth Layers

Layer 1: NETWORK

- HTTPS/TLS 1.2+ (all traffic encrypted)
- CORS whitelist (only frontend origin)
- Rate limiting (1000 req/min global, stricter on auth)

Layer 2: APPLICATION

- Helmet.js (security headers: HSTS, X-Frame-Options, CSP)
- Input validation (class-validator DTOs, whitelist mode)
- SQL injection prevention (Prisma parameterized queries)
- XSS prevention (no HTML rendering from user input)

Layer 3: AUTHENTICATION

- Password hashing (Argon2id)
- JWT tokens (short-lived access, server-stored refresh)
- Token rotation on refresh
- Brute-force protection (rate limiting on login)

Layer 4: AUTHORIZATION

- Role-Based Access Control (CUSTOMER, ADMIN, SUPER_ADMIN)
- Row-level security (users access own data only)
- Guard-based endpoint protection
- Decorator-based role requirements

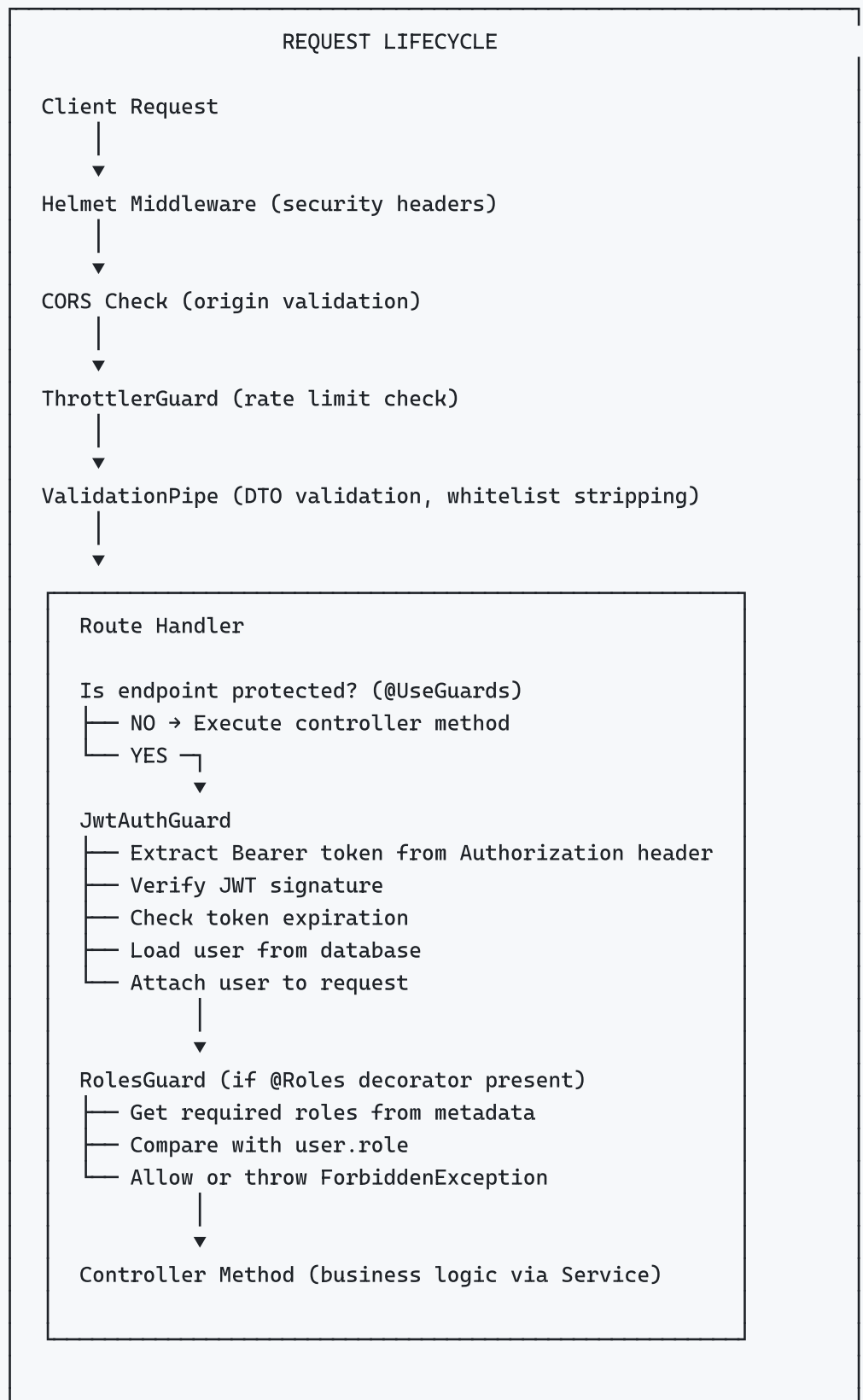
Layer 5: DATA

- Encrypted at rest (cloud provider)
- Encrypted in transit (TLS)
- Sensitive fields hashed (passwords)
- PCI-DSS compliance (Stripe handles card data)

Layer 6: AUDIT

- Order status change history
- Loyalty transaction log
- Application logs (errors, requests)

5.2 Authentication Architecture



6. API Architecture

6.1 URL Namespace

/api	
├── /auth	← Authentication (public + protected)
├── /account	← User account management (protected)
├── /products	← Product catalog (public + admin)
├── /categories	← Categories (public + admin)
├── /brands	← Brands (public + admin)
├── /cart	← Shopping cart (protected)
├── /orders	← Order management (protected)
├── /favorites	← Wishlist (protected)
├── /promo-codes	← Promotions (protected + admin)
├── /stripe	← Payments (public + protected + admin)
├── /media	← File uploads (admin)
├── /content	← CMS content (public + admin)
├── /cms	← CMS pages (public + admin)
├── /blog	← Blog (public + admin)
├── /navigation	← Menu system (public + admin)
├── /collections	← Curated lists (public + admin)
├── /settings	← Site settings (public + admin)
├── /admin	← Admin dashboard (admin only)
│ ├── /stats	← Dashboard KPIs
│ ├── /orders	← Order management
│ ├── /customers	← Customer management
│ └── /reports	← Analytics & reports
└── /uploads/*	← Static file serving (public, no /api prefix)

6.2 API Response Contract

Success (200/201):

```
{
  "data": { ... } | [ ... ],
  "meta": {                                // For paginated responses
    "total": 805,
    "page": 1,
    "limit": 20,
    "totalPages": 41
  }
}
```

Error (4xx/5xx):

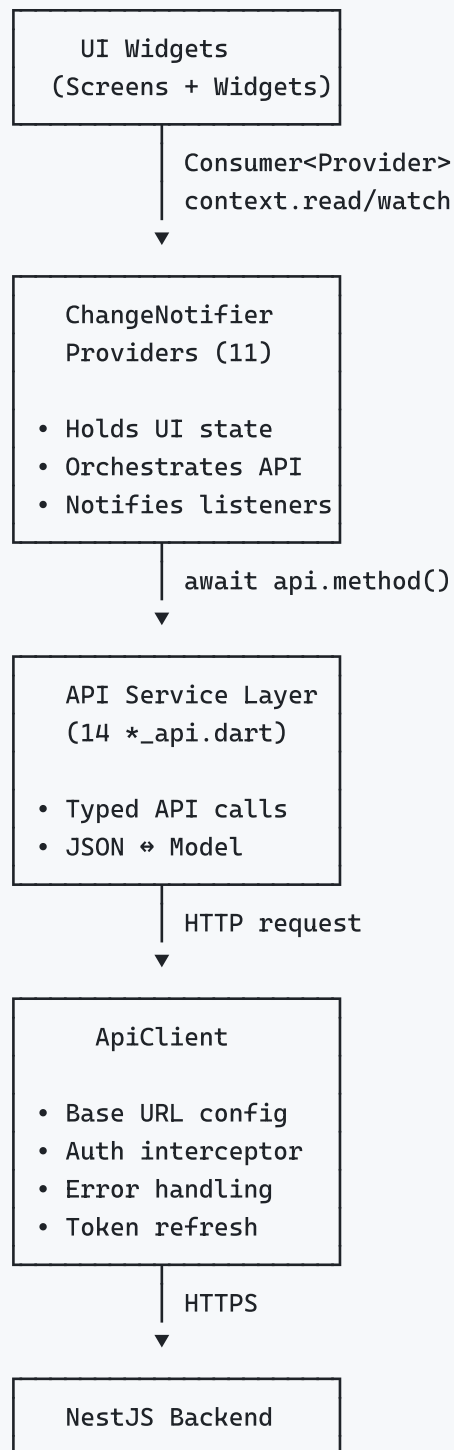
```
{
  "statusCode": 400,
  "message": "Validation failed" | ["field must be string"],
  "error": "Bad Request"
}
```

6.3 Endpoint Summary

Access Level	Count	Description
Public	34	Product browsing, CMS, blog, navigation
Protected (Customer)	34	Cart, orders, favorites, account, payments
Admin Only	100+	CRUD operations, reports, settings, media
Total	170+	Complete API surface

7. Frontend Architecture

7.1 State Management Pattern



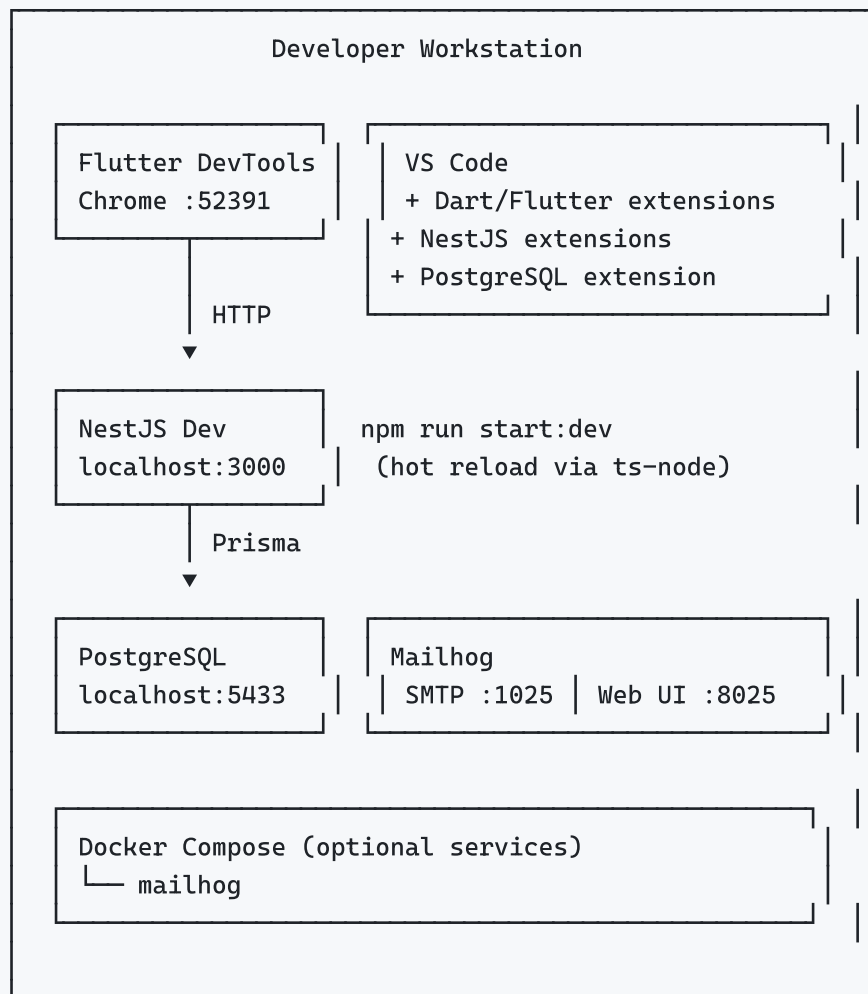
7.2 Navigation Architecture

MaterialApp

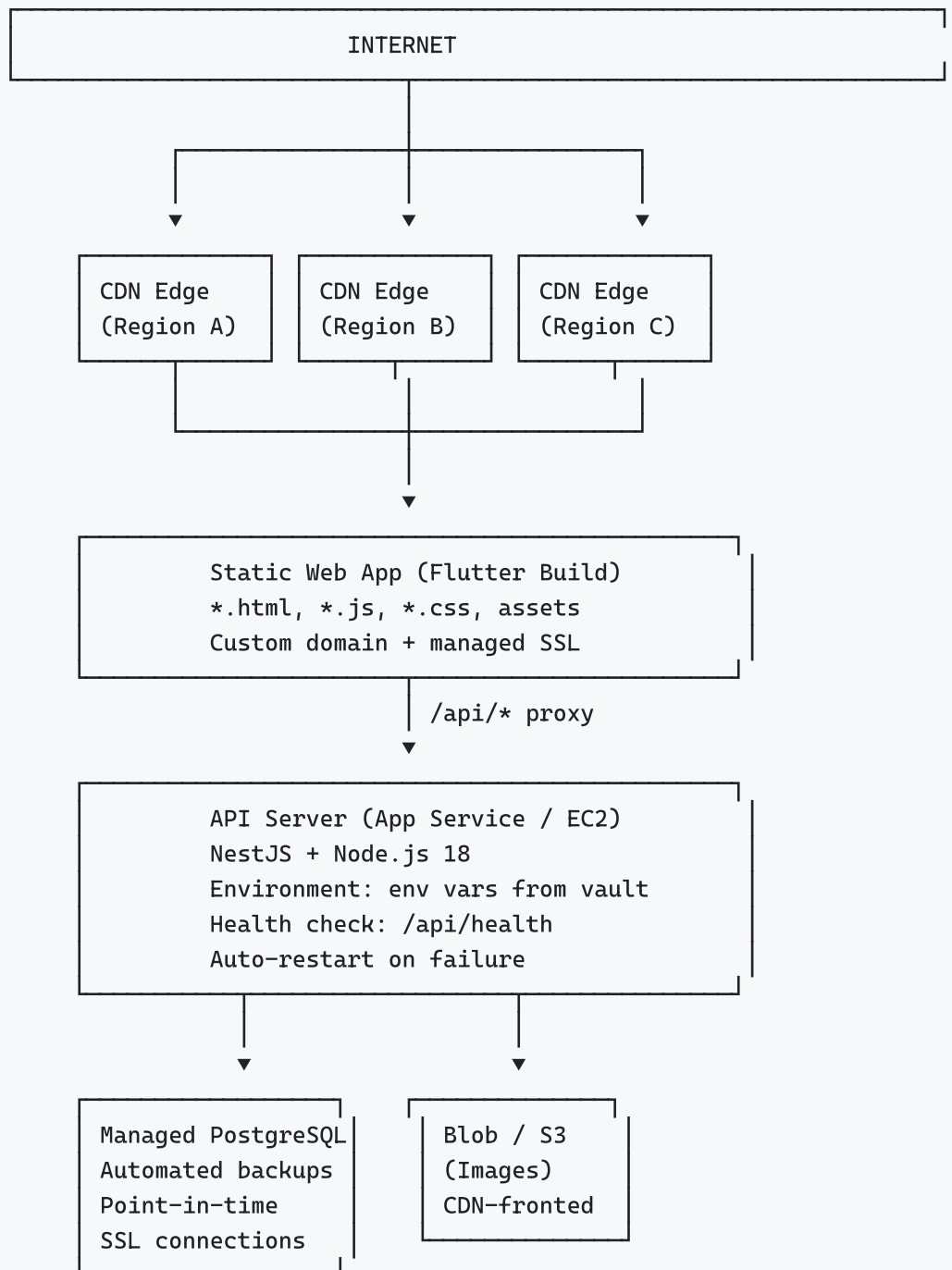
/ (home)	HomeScreen / HomeScreenCms
/login	LoginScreen
/signup	SignupScreen
/forgot-password	ForgotPasswordScreen
/verify-email	VerifyEmailScreen
/products	CategoryScreen (product listing)
/product/:id	ProductDetailScreen
/category/:id	CategoryLandingScreen
/search	SearchScreen
/cart	CartScreen
/checkout	CheckoutScreen
/favorites	FavoritesScreen
/account	MyAccountScreen (with sub-routes)
/addresses	MyAddressesScreen
/orders	Order History
/loyalty	LoyaltyProgramScreen
/about	AboutUsScreen
/bulk-order	BulkOrderScreen
/admin	Admin Routes (protected by role check)
/admin/dashboard	AdminDashboardScreen
/admin/products	AdminProductsScreen
/admin/categories	AdminCategoriesScreen
/admin/brands	AdminBrandsScreen
/admin/orders	AdminOrdersScreen
/admin/customers	AdminCustomersScreen
/admin/banners	AdminBannersScreen
/admin/promos	AdminPromoCodesScreen
/admin/landing	AdminLandingPagesScreen
/admin/reports	AdminReportsScreen
/admin/stripe	AdminStripeConfigScreen
/admin/vat	AdminVatConfigScreen

8. Infrastructure Architecture

8.1 Development Environment



8.2 Production Architecture



9. Build & Deployment Pipeline

9.1 CI/CD Flow

Developer Push to GitHub



GitHub Actions Trigger

Backend Pipeline:

- Install dependencies (npm ci)
- Lint (ESLint)
- Build (tsc → dist/)
- Unit tests (Jest)
- E2E tests (Supertest)
- Deploy to App Service / EC2

Frontend Pipeline:

- Install dependencies (flutter pub get)
- Analyze (flutter analyze)
- Build (flutter build web --release)
- Deploy to Static Web Apps / S3

9.2 Environment Configuration

Variable	Development	Production
DATABASE_URL	localhost:5433	Managed DB connection string
NODE_ENV	development	production
JWT_ACCESS_SECRET	dev-secret	32+ char random
JWT_REFRESH_SECRET	dev-secret	32+ char random
FRONTEND_URL	http://localhost:5000	https://solo-ecommerce.com
STRIPE_SECRET_KEY	sk_test_xxx	sk_live_xxx
SMTP_HOST	localhost	smtp.sendgrid.net
UPLOAD_PATH	./uploads	/app/uploads or cloud storage

10. Performance Architecture

10.1 Optimization Strategies

Layer	Strategy	Implementation
CDN	Cache static assets	Flutter build output served from edge
API	Response caching headers	Cache-Control for product lists

Layer	Strategy	Implementation
Database	Strategic indexing	Indexes on FK columns, email, slug
Images	Optimized uploads	WebP conversion, max 1920px width
Frontend	Lazy loading	Provider-based on-demand data loading
Queries	Batch operations	resolveProductImageUrls batches DB lookups
Pagination	Cursor/offset	20 items per page default

10.2 Scalability Path

Phase 1 (Current): Single Server

└─ App Service B1 + Flexible Server B1ms

Phase 2 (Growth): Vertical Scaling

└─ App Service S1 + Flexible Server B2s + Redis Cache

Phase 3 (Scale): Horizontal Scaling

└─ Multiple API instances + Load Balancer
+ Read Replicas + CDN + Background Workers

11. Monitoring & Observability

Concern	Tool	What's Monitored
Health Check	/api/health endpoint	Server alive, DB connected
Error Tracking	Application Insights / CloudWatch	Unhandled exceptions, 5xx rates
Performance	APM metrics	Response times, throughput, p95 latency
Availability	Uptime monitoring	HTTP 200 checks every 60s
Logs	Structured logging	Request/response, auth events, order events
Alerts	Threshold-based	Error rate > 1%, latency > 2s, disk > 90%

12. Disaster Recovery

Scenario	RTO	RPO	Recovery Strategy
API server crash	< 5 min	0	Auto-restart via health check

Scenario	RTO	RPO	Recovery Strategy
Database corruption	< 1 hour	< 1 hour	Point-in-time restore from backup
Region outage	< 4 hours	< 1 hour	Redeploy to alternate region
Data breach	< 1 hour	—	Rotate secrets, revoke tokens, notify users

End of Architecture Document